

DOCUMENTACAO TECNICA COMPLETA

Sistema de Controle de Revisoes de Veiculos

Django REST Framework | PostgreSQL 15 | Vue 3 | Docker Compose

Schema: jose_gabriel | Nivel Junior | Teste Pratico

Autor: Jose Gabriel | Versao 1.0 | 2026

1. Visao Geral do Projeto

O Sistema de Controle de Revisoes de Veiculos e uma aplicacao web completa desenvolvida como teste pratico de nivel junior. Ele permite o cadastro e gerenciamento de proprietarios de veiculos, o registro do historico de revisoes mecanicas e a geracao de relatorios analiticos, incluindo a previsao de proximas revisoes com base no historico de cada proprietario.

O sistema segue a arquitetura cliente-servidor com comunicacao exclusiva via API REST. O backend expoe endpoints HTTP que o frontend consome usando o cliente HTTP axios. Nao ha renderizacao server-side — toda a logica de interface roda no navegador do usuario.

1.1 Funcionalidades implementadas

Cadastros (CRUD completo)

- Proprietarios: criacao, listagem, edicao parcial e exclusao em cascata
- Veiculos: suporte a 4 tipos (Carro, Moto, Triciclo, Caminhao) com campos especificos por tipo
- Revisoes: lancamento, listagem, edicao e exclusao com dados do veiculo e proprietario

Relatorios analiticos

- Veiculos por proprietario ordenado por nome
- Ranking de marcas por numero de veiculos
- Totais de marcas separados por genero do proprietario
- Genero com maior numero de veiculos cadastrados
- Proprietarios separados por genero com idade media calculada
- Revisoes dentro de um periodo especificado
- Marcas com maior numero de revisoes realizadas
- Pessoas com maior numero de revisoes
- Media de dias entre revisoes consecutivas de cada pessoa (usa LAG do PostgreSQL)
- Previsao das proximas revisoes com base no intervalo medio (usa CTEs encadeadas)

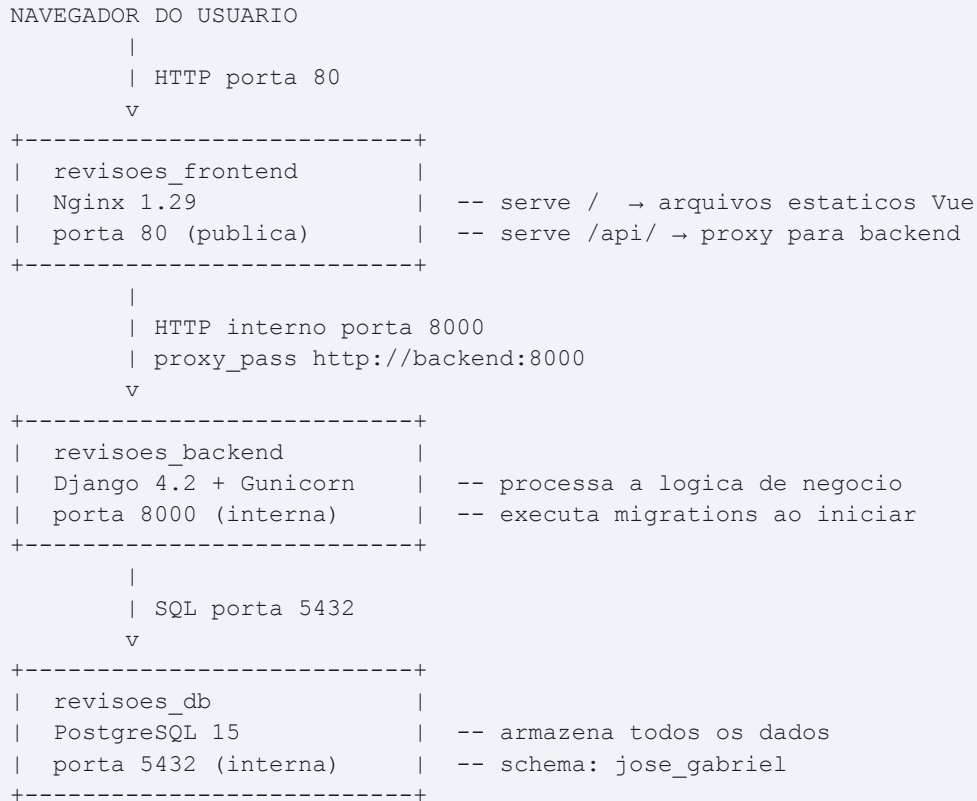
1.2 Tecnologias e versoes

Camada	Tecnologia	Versao	Funcao no projeto
Backend	Python	3.12	Linguagem principal do servidor
Backend	Django	4.2+	Framework MVC — models, views, migrations
Backend	Django REST	3.14+	Serializers, decorators e respostas JSON

Camada	Tecnologia	Versao	Funcao no projeto
Backend	Gunicorn	21+	Servidor WSGI de producao (substitui runserver)
Backend	python-decouple	3.8+	Le variaveis do .env — separa config do codigo
Backend	django-cors-headers	4.0+	Permite requisicoes cross-origin do Vue
Banco	PostgreSQL	15	Banco relacional com schema personalizado
Banco	psycopg2-binary	2.9+	Driver Python para conectar ao PostgreSQL
Frontend	Vue 3	3.x	Framework JS reativo — Composition API
Frontend	Vite	5+	Bundler e servidor de desenvolvimento
Frontend	Vue Router	4.x	Navegacao SPA entre paginas
Frontend	Axios	1.x	Cliente HTTP para consumir a API REST
Infra	Docker	26+	Containerizacao dos servicos
Infra	Docker Compose	v2	Orquestracao dos 3 containers
Infra	Nginx	1.29	Serve o frontend e faz proxy para o Django

2. Arquitetura do Sistema

O sistema é composto por 3 containers Docker que se comunicam em uma rede interna privada chamada 'rede_app'. O usuário acessa apenas a porta 80 do container frontend — as demais portas são internas.



2.1 Fluxo de uma requisicao

Exemplo: usuário clica em 'Novo proprietario' e preenche o formulário.

1. Usuário clica no botão — Vue chama `abrirModalCriar()`
2. Usuário preenche o formulário e confirma — Vue chama `salvar()`
3. `axios.post('/api/users/creat/', dados)` envia HTTP POST para o Nginx
4. Nginx recebe na porta 80 e faz `proxy_pass` para `backend:8000`
5. Gunicorn recebe e repassa para o Django
6. Django executa `creat_Usuario()` — valida via `UsuarioSerializer`
7. Serializer valida os campos (CPF único, campos obrigatórios etc.)
8. Se válido: ORM executa `INSERT INTO jose_gabriel.usuario`
9. PostgreSQL persiste o registro e retorna o objeto criado
10. Django serializa o objeto para JSON e retorna HTTP 201 Created

11. Vue recebe a resposta — atualiza a lista local e fecha o modal

2.2 Estrutura completa de arquivos

```
Site_Carros/
├── .env                                <- credenciais para o Docker Compose
├── docker-compose.yml                 <- define os 3 containers e a rede
├── Back_End/
│   ├── .env                          <- credenciais para o Django local
│   ├── Dockerfile                    <- image: python:3.12-slim + pip + gunicorn
│   ├── requirements.txt               <- dependencias Python instaladas no container
│   ├── manage.py                     <- CLI do Django
│   ├── populate.py                   <- script para popular o banco com dados de teste
│   ├── Back_End/
│   │   ├── settings.py               <- modulo principal do projeto Django
│   │   ├── urls.py                   <- toda a configuracao do Django
│   │   ├── wsgi.py                   <- inclui as rotas de api/
│   │   └── asgi.py                   <- ponto de entrada para o Gunicorn
│   ├── api/
│   │   ├── views.py                  <- app Django com os endpoints REST
│   │   ├── serializers.py            <- funcoes de cada endpoint
│   │   ├── urls.py                   <- conversao Model <-> JSON
│   │   └── migrations/
│   │       └── 0000_create_schema.py <- cria o schema jose_gabriel
│   └── apps/
│       └── core/
│           └── models.py              <- todos os models do banco
├── front-end/
│   ├── Dockerfile                    <- stage 1: Node (build) | stage 2: Nginx
│   ├── nginx.conf                    <- serve static + proxy /api/
│   ├── vite.config.js                <- alias @ para src/ e config do Vite
│   ├── package.json                  <- dependencias Node (vue, vue-router, axios)
│   └── src/
│       ├── main.js                   <- cria o app Vue, registra router, monta #app
│       ├── App.vue                   <- componente raiz: navbar + router-view
│       ├── style.css                 <- importa os 6 modulos CSS em ordem
│       ├── router/
│       │   └── index.js               <- mapa: URL -> componente Vue
│       ├── services/
│       │   └── api.js                 <- instancia axios com baseURL configurada
│       ├── assets/styles/
│       │   ├── global.css             <- CSS modular separado por responsabilidade
│       │   ├── :root com variaveis, reset, tipografia
│       │   ├── nav.css                <- barra de navegacao
│       │   ├── paginas.css            <- layout das paginas
│       │   └── tabelas.css             <- tabelas, badges, tfoot
```

```
|   ├── formularios.css    <- inputs, selects, botoes, filtros
|   └── componentes.css   <- cards, modais, alertas
└── views/                <- uma pagina por rota
    ├── HomeView.vue      <- dashboard com totais e proximas revisoes
    ├── UsuariosView.vue  <- CRUD de proprietarios com modal
    ├── VeiculosView.vue  <- CRUD com filtro e form dinamico por tipo
    └── RevisoesView.vue  <- CRUD com filtro por texto e datas
```

3. Banco de Dados

O PostgreSQL organiza objetos em schemas, que funcionam como namespaces. Todas as tabelas do projeto estão dentro do schema `jose_gabriel`, isolado do schema public padrão. Isso evita conflitos de nomes em ambientes compartilhados e é uma boa prática em bancos multi-tenant.

3.1 Como o schema é criado

A migration `0000_create_schema.py` roda automaticamente na primeira execução do Docker e garante que o schema existe antes das migrations dos models:

```
# Back_End/api/migrations/0000_create_schema.py
from django.db import migrations

class Migration(migrations.Migration):
    initial = True
    dependencies = []
    operations = [
        migrations.RunSQL(
            sql='CREATE SCHEMA IF NOT EXISTS jose_gabriel;',
            reverse_sql='DROP SCHEMA IF EXISTS jose_gabriel CASCADE;'
        )
    ]
```

O `IF NOT EXISTS` garante idempotência — rodar a migration múltiplas vezes não causa erro.

3.2 Diagrama completo de relacionamentos

```
jose_gabriel.usuario
PK id BIGINT AUTO
  cpf VARCHAR(14) UNIQUE NOT NULL
  nome VARCHAR(255) NOT NULL
  genero CHAR(1) CHECK (M, F)
  data_nascimento DATE NOT NULL
  endereco VARCHAR(255) NULL
  |
  | FK proprietario_id CASCADE DELETE
  | 1 usuario -> N veiculos
v

jose_gabriel.veiculo (tabela BASE da heranca)
PK id BIGINT AUTO
FK proprietario_id -> usuario.id CASCADE
  placa VARCHAR(10) UNIQUE NOT NULL
  marca VARCHAR(50) NOT NULL
  modelo VARCHAR(50) NOT NULL
  ano INTEGER NOT NULL
  |
```

```

| OneToOne reverse (heranca multi-tabela Django)
|-- 1:1 --> jose_gabriel.carro
|
|         ar_condicionado  BOOLEAN
|         numero_portas    INTEGER
|         tipo_combustivel  VARCHAR(50)
|
|-- 1:1 --> jose_gabriel.moto
|
|         refrigeracao  VARCHAR(50)
|         tipo_partida   VARCHAR(50)
|         cilindradas    INTEGER
|
|-- 1:1 --> jose_gabriel.triciclo
|
|         tipo_tracao     VARCHAR(50)
|         capacidade_carga DECIMAL(10,2)
|
|-- 1:1 --> jose_gabriel.caminhao
|
|         quantidade_eixos    INTEGER
|         capacidade_toneladas DECIMAL(10,2)
|         tipo_carroceria     VARCHAR(100)
|
| FK veiculo_id CASCADE DELETE
| 1 veiculo -> N revisoes
v
jose_gabriel.revisao
PK id BIGINT AUTO
FK veiculo_id -> veiculo.id CASCADE
data_revisao DATE NOT NULL
quilometragem DECIMAL(10,2) NOT NULL
descricao TEXT NOT NULL
custo DECIMAL(10,2) NOT NULL
responsavel VARCHAR(100) NOT NULL

```

3.3 Heranca multi-tabela no Django

A heranca multi-tabela e um padrao do Django em que o model filho herda do pai. O Django cria automaticamente uma FK OneToOne entre a tabela filha e a tabela pai. Isso permite:

- Buscar todos os veiculos independente do tipo via `Veiculo.objects.all()`
- Acessar campos especificos do tipo via `veiculo.carro.numero_portas`
- Identificar o tipo verificando a existencia do atributo (`hasattr`)

```

# Como o serializer identifica o tipo do veiculo
def get_tipo(self, obj):
    # hasattr verifica se existe a FK reversa da tabela filha
    if hasattr(obj, 'carro'): return 'Carro'
    if hasattr(obj, 'moto'):  return 'Moto'
    if hasattr(obj, 'triciclo'): return 'Triciclo'
    if hasattr(obj, 'caminhao'): return 'Caminhao'
    return 'Veiculo'

# Como criar um Carro (insere em veiculo E em carro)

```



```
Carro.objects.create(
    proprietario=usuario, # FK para usuario
    placa='ABC-1234',      # campo da tabela veiculo
    marca='Honda',
    modelo='Civic',
    ano=2022,
    numero_portas=4,        # campo da tabela carro
    tipo_combustivel='Flex',
    ar_condicionado=True,
)
# Django executa: INSERT INTO veiculo (...) e INSERT INTO carro (...)
```

3.4 Problema N+1 e como foi resolvido

O problema N+1 ocorre quando o código faz 1 query para buscar N objetos e depois N queries adicionais para acessar dados relacionados. Com 100 veículos, isso seria 101 queries. O projeto resolve isso com `select_related` e `prefetch_related`:

```
# ERRADO - problema N+1: 1 + N queries
veiculos = Veiculo.objects.all()
for v in veiculos:
    print(v.proprietario.nome) # query extra para cada veiculo

# CORRETO - select_related: sempre 1 query com JOIN
veiculos = Veiculo.objects.select_related('proprietario').all()

# Para relacoes aninhadas (usuario -> veiculos -> revisoes)
# prefetch_related: 3 queries totais independente da quantidade
usuario = Usuario.objects.prefetch_related('veiculos__revisoes').get(pk=pk)
```

4. Backend — Django REST Framework

4.1 Settings — configuracoes criticas

O arquivo settings.py usa python-decouple para ler variaveis do .env. Isso mantem credenciais fora do codigo e permite configuracoes diferentes entre Docker e desenvolvimento local:

```
# Back_End/Back_End/settings.py

from decouple import config

# Segredo do Django - nunca hardcoded no codigo
SECRET_KEY = config('SECRET_KEY')
DEBUG      = config('DEBUG', default=False, cast=bool)

# Hosts permitidos - vem do .env separados por virgula
ALLOWED_HOSTS = str(config('DJANGO_ALLOWED_HOSTS', default='localhost')).split(',')

# Banco de dados - HOST muda conforme o ambiente
# Docker:  DB_HOST=db          (nome do container)
# Local:   DB_HOST=localhost
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME':    config('DB_NAME'),
        'USER':    config('DB_USER'),
        'PASSWORD': config('DB_PASSWORD'),
        'HOST':    config('DB_HOST', default='localhost'),
        'PORT':    config('DB_PORT', default='5432'),
    }
}

# CORS - lista fixa, NAO use variaveis de ambiente aqui
# django-cors-headers valida o formato e rejeita strings invalidas
CORS_ALLOWED_ORIGINS = [
    'http://localhost',
    'http://127.0.0.1',
]

# CSRF - necessario para POST/PUT/DELETE via proxy (Nginx/HTTPS)
CSRF_TRUSTED_ORIGINS = [
    'http://localhost',
    'http://127.0.0.1',
]

# corsheaders DEVE ser o primeiro middleware da lista
MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware', # primeiro
    'django.middleware.security.SecurityMiddleware',
    ...
]
```

4.2 Serializers — conversao Model <-> JSON

O serializer e a camada que converte objetos Python (instancias de Model) em JSON para a resposta HTTP, e JSON recebido em objetos Python para salvar no banco. Ele tambem executa validacao dos dados recebidos.

```
# Campos calculados via SerializerMethodField
# Nao existem no banco - sao calculados na hora da serializacao
class UsuarioSerializer(serializers.ModelSerializer):
    idade = serializers.SerializerMethodField()
    genero_display = serializers.SerializerMethodField()
    total_veiculos = serializers.SerializerMethodField()

    def get_idade(self, obj):
        hoje = date.today()
        return hoje.year - obj.data_nascimento.year - (
            (hoje.month, hoje.day) < (obj.data_nascimento.month,
            obj.data_nascimento.day)
        )

    def get_genero_display(self, obj):
        return 'Masculino' if obj.genero == 'M' else 'Feminino'

    def get_total_veiculos(self, obj):
        # obj.veiculos e o related_name da FK em Veiculo
        return obj.veiculos.count()
```

O serializer aninhado UsuarioDetalheSerializer retorna toda a arvore em uma unica requisicao:

```
# Hierarquia de serializacao aninhada
UsuarioDetalheSerializer
├── veiculos: VeiculoComRevisoesSerializer (many=True)
│   └── revisoes: RevisaoResumidaSerializer (many=True)

# Resultado JSON:
{
  'id': 1, 'nome': 'Carlos', 'idade': 38,
  'veiculos': [
    {
      'placa': 'ABC-1234', 'marca': 'Honda', 'tipo': 'Carro',
      'revisoes': [
        { 'data_revisao': '2024-01-10', 'custo': '350.00' }
      ]
    }
  ]
}
```

4.3 Views — padroes adotados

Todos os endpoints seguem o mesmo padrao de tratamento de erros, usando `get_object_or_404` para evitar erros 500, e `partial=True` nos PUTs para permitir atualizacão parcial:

```
# Padrao de uma view de edicao (PUT)
@api_view(['PUT'])
def update_Usuario(request: Request, pk: int):
    # get_object_or_404 retorna 404 automaticamente se nao existir
    # Sem isso, .get() lancaria DoesNotExist causando erro 500
    usuario = get_object_or_404(Usuario, pk=pk)

    # partial=True: aceita apenas os campos enviados
    # Sem isso: todos os campos obrigatorios seriam exigidos no PUT
    serializer = UsuarioSerializer(usuario, data=request.data, partial=True)

    if serializer.is_valid():
        serializer.save()
        return Response(serializer.data, status=status.HTTP_200_OK)

    # Retorna os erros de validacao com detalhe por campo
    # Ex: { 'cpf': ['CPF ja cadastrado.'] }
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

4.4 Todos os endpoints da API

USUARIOS — /api/users/

Metodo	Rota	Status sucesso	Descricao
GET	/api/users/	200 OK	Lista todos com idade e total de veiculos
GET	/api/users/{id}/	200 OK	Detalhe com veiculos e revisoes aninhados
POST	/api/users/creat/	201 Created	Cria novo proprietario
PUT	/api/users/{id}/update/	200 OK	Edita parcialmente (envia so o que mudou)
DELETE	/api/users/{id}/delete/	200 OK	Remove com cascata em veiculos e revisoes
GET	/api/users/relatorio/genero/	200 OK	Total e idade media por genero

VEICULOS — /api/veiculos/

Metodo	Rota	Status sucesso	Descricao
GET	/api/veiculos/	200 OK	Lista todos com nome do proprietario e tipo
POST	/api/veiculos/creat/	201 Created	Cadastra veiculo com campos do tipo
PUT	/api/veiculos/{id}/update/	200 OK	Edita campos base do veiculo
DELETE	/api/veiculos/{id}/delete/	200 OK	Remove com cascata em revisoes
GET	/api/veiculos/relatorio/proprietario/	200 OK	Veiculos ordenados por nome do dono
GET	/api/veiculos/relatorio/marcas/	200 OK	Ranking de marcas por quantidade
GET	/api/veiculos/relatorio/marcas_genero/	200 OK	Totais de marcas separados por genero
GET	/api/veiculos/relatorio/genero/	200 OK	Qual genero tem mais veiculos

REVISOES — /api/revisoes/

Metodo	Rota	Status sucesso	Descricao
GET	/api/revisoes/	200 OK	Lista todas com placa e proprietario
POST	/api/revisoes/creat/	201 Created	Lanca nova revisao
PUT	/api/revisoes/{id}/update/	200 OK	Edita revisao existente
DELETE	/api/revisoes/{id}/delete/	200 OK	Remove revisao
GET	/api/revisoes/relatorio/periodo/	200 OK	Revisoes entre ?inicio= e ?fim= (YYYY-MM-DD)
GET	/api/revisoes/relatorio/marcas/	200 OK	Marcas com mais revisoes
GET	/api/revisoes/relatorio/pessoas/	200 OK	Pessoas com mais revisoes

Metodo	Rota	Status sucesso	Descricao
GET	/api/revisoes/relatorio/media_tempo/	200 OK	Media de dias entre revisoes por pessoa
GET	/api/revisoes/relatorio/proximas/	200 OK	Proximas revisoes previstas

5. Consultas SQL dos Relatorios

Os relatorios simples usam o ORM do Django — mais seguro, portavel e legivel. Os relatorios avancados que dependem de window functions exclusivas do PostgreSQL usam Raw SQL com `cursor.execute()`.

5.1 ORM — relatorios simples

Usuarios por genero com idade media

```
# ORM
Usuario.objects.values('genero').annotate(
    total = Count('id'),
    idade_media = Avg(
        ExpressionWrapper(
            date.today().year - F('data_nascimento__year'),
            output_field=FloatField()
        )
    )
)

-- SQL equivalente gerado pelo Django
SELECT genero,
       COUNT(id) AS total,
       AVG(EXTRACT(YEAR FROM NOW()) - EXTRACT(YEAR FROM data_nascimento))
       AS idade_media
FROM jose_gabriel.usuario
GROUP BY genero
ORDER BY total DESC;
```

Ranking de marcas por numero de veiculos

```
-- SQL equivalente
SELECT marca, COUNT(id) AS total
FROM jose_gabriel.veiculo
GROUP BY marca
ORDER BY total DESC;
```

Pessoas com mais revisoes

```
-- SQL equivalente (dois JOINS para chegar ao usuario)
SELECT u.nome, u.genero, COUNT(r.id) AS total
FROM jose_gabriel.revisao r
JOIN jose_gabriel.veiculo v ON v.id = r.veiculo_id
JOIN jose_gabriel.usuario u ON u.id = v.proprietario_id
GROUP BY u.id, u.nome, u.genero
ORDER BY total DESC;
```

5.2 Raw SQL — relatorios avancados (window functions)

O ORM do Django suporta window functions basicas, mas a combinacao de LAG() com GROUP BY e a CTE com DISTINCT ON fica ilegivel via ORM. O Raw SQL e mais claro e eficiente nestes casos.

Media de dias entre revisoes consecutivas

LAG() retorna o valor da linha anterior dentro da particao. Aqui calcula a data da revisao anterior da mesma pessoa para obter o intervalo em dias.

```
SELECT
    u.nome,
    ROUND(
        AVG(
            -- diferenca entre data atual e data anterior da mesma pessoa
            r.data_revisao - LAG(r.data_revisao) OVER (
                PARTITION BY u.id          -- separa por pessoa
                ORDER BY r.data_revisao -- ordena cronologicamente
            )
        )::numeric, 1
    ) AS media_dias
FROM jose_gabriel.revisao r
JOIN jose_gabriel.veiculo v ON v.id = r.veiculo_id
JOIN jose_gabriel.usuario u ON u.id = v.proprietario_id
GROUP BY u.id, u.nome
ORDER BY media_dias ASC NULLS LAST;

-- NULLS LAST: pessoas com apenas 1 revisao ficam no final
-- (LAG retorna NULL para a primeira revisao de cada pessoa)
```

Proximas revisoes previstas — 3 CTEs encadeadas

CTE 1 calcula os intervalos com LAG. CTE 2 agrega a media. CTE 3 usa DISTINCT ON (exclusivo PostgreSQL) para pegar a revisao mais recente de cada pessoa sem subquery correlated.

```
WITH

-- CTE 1: busca a revisao anterior de cada pessoa usando LAG
intervalos AS (
    SELECT
        u.id          AS usuario_id,
        u.nome,
        r.data_revisao,
        LAG(r.data_revisao) OVER (
            PARTITION BY u.id ORDER BY r.data_revisao
        ) AS revisao_anterior
    FROM jose_gabriel.revisao r
    JOIN jose_gabriel.veiculo v ON v.id = r.veiculo_id
    JOIN jose_gabriel.usuario u ON u.id = v.proprietario_id
```



```

),

-- CTE 2: calcula a media de dias por pessoa
-- WHERE exclui a primeira revisao (revisao_anterior e NULL)
media_por_usuario AS (
    SELECT usuario_id, nome,
           AVG(data_revisao - revisao_anterior) AS media_dias
    FROM intervalos
    WHERE revisao_anterior IS NOT NULL
    GROUP BY usuario_id, nome
),

-- CTE 3: pega apenas a ultima revisao de cada pessoa
-- DISTINCT ON e exclusivo do PostgreSQL:
-- retorna 1 linha por (proprietario_id), a mais recente pela ORDER BY
ultima_revisao AS (
    SELECT DISTINCT ON (v.proprietario_id)
           v.proprietario_id AS usuario_id,
           r.data_revisao     AS ultima
    FROM jose_gabriel.revisao r
    JOIN jose_gabriel.veiculo v ON v.id = r.veiculo_id
    ORDER BY v.proprietario_id, r.data_revisao DESC
)

-- Query final: soma a media de dias a ultima revisao
SELECT
    m.nome,
    ROUND(m.media_dias::numeric, 0)                AS media_dias,
    u.ultima                                       AS ultima_revisao,
    (u.ultima + (ROUND(m.media_dias::numeric, 0) || ' days')
     ::interval)::date                           AS proxima_revisao
FROM media_por_usuario m
JOIN ultima_revisao u ON u.usuario_id = m.usuario_id
ORDER BY proxima_revisao ASC;

```

6. Frontend — Vue 3

O frontend é uma Single Page Application (SPA). O navegador carrega um único HTML e o Vue substitui o conteúdo da tela dinamicamente conforme o usuário navega — sem recarregar a página.

6.1 Como o Vue inicializa

```
// 1. index.html - unico HTML do projeto
<div id='app'></div>  <!-- Vue monta aqui -->
<script type='module' src='/src/main.js'></script>

// 2. main.js - ponto de entrada JavaScript
import { createApp } from 'vue'
import App    from './App.vue'      // componente raiz
import router from './router'       // sistema de rotas
import './style.css'                // importa os 6 modulos CSS

createApp(App) // cria o app com App.vue como raiz
  .use(router) // registra o Vue Router
  .mount('#app') // injeta no div#app do HTML

// 3. App.vue - layout fixo: navbar + espaco para a pagina
<nav class='navbar'>
  <router-link to='/'>Home</router-link>
  <router-link to='/usuarios'>Usuarios</router-link>
  ...
</nav>
<router-view />  <!-- pagina atual renderizada aqui -->
```

6.2 Sistema de rotas — router/index.js

O Vue Router mapeia cada URL para um componente .vue. Ao clicar em um router-link ou chamar router.push(), o Router troca o componente dentro do router-view sem requisição HTTP ao servidor.

URL	Componente	O que a página faz
/	HomeView.vue	Dashboard: totais + atalhos + próximas revisões
/usuarios	UsuariosView.vue	Tabela com CRUD completo via modal
/veiculos	VeiculosView.vue	Tabela com filtro local + form dinâmico por tipo
/revisoes	RevisoesView.vue	Tabela com filtro texto/datas + total de custos

6.3 Composition API — padrões usados

```
// ref() - dado reativo: mudança atualiza a tela automaticamente
const usuarios = ref([]) // lista da API
const carregando = ref(true) // controla estado de loading
const mensagem = ref(null) // feedback ao usuario

// computed() - valor derivado, recalcula quando dependências mudam
// Não faz nova requisição HTTP - filtra em memória
const veiculosFiltrados = computed(() => {
  return veiculos.value.filter(v => {
    const txt = filtroTexto.value.toLowerCase()
    return !txt || v.placa.toLowerCase().includes(txt)
  })
})

// onMounted() - executa quando o componente monta no DOM
// Equivalente ao created() do Vue 2
onMounted(async () => {
  const { data } = await api.get('users/')
  usuarios.value = data
  carregando.value = false
})

// Promise.all - requisições em paralelo (mais rápido que sequencial)
// Tempo total = mais lenta das 4, não soma das 4
const [resUsuarios, resVeiculos, resRevisoes, resProximas] = await Promise.all([
  api.get('users/'),
  api.get('veiculos/'),
  api.get('revisoes/'),
  api.get('revisoes/relatorio/proximas/'),
])
```

6.4 CSS modular — variáveis globais

Todo o CSS fica em arquivos separados importados pelo style.css central. Nenhum arquivo .vue tem bloco style. Para mudar a cor primária do sistema inteiro, basta alterar uma variável:

```
/* src/assets/styles/global.css */
:root {
  --cor-primaria: #1E3A5F; /* mude aqui para alterar o sistema inteiro */
  --cor-secundaria: #2E6DA4;
  --cor-destaque: #27AE60; /* verde - botoes de sucesso */
  --cor-perigo: #E74C3C; /* vermelho - botoes de excluir */
  --raio-borda: 8px;
  --sombra: 0 2px 8px rgba(0,0,0,0.08);
  --fonte: 'Segoe UI', Arial, sans-serif;
}

/* src/style.css - centraliza as importações */
@import './assets/styles/global.css';
@import './assets/styles/nav.css';
```

```
@import './assets/styles/paginas.css';  
@import './assets/styles/tabelas.css';  
@import './assets/styles/formularios.css';  
@import './assets/styles/componentes.css';
```

7. Docker — Infraestrutura

7.1 Dockerfile do backend

Usa python:3.12-slim para menor tamanho de image. O comando CMD roda as migrations automaticamente antes de iniciar o Gunicorn — garantindo que o banco esteja sempre atualizado:

```
# Back_End/Dockerfile
FROM python:3.12-slim

WORKDIR /app

# Copia requirements primeiro — aproveita cache do Docker
# Se o requirements nao mudou, esta layer usa cache
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

# sh -c permite encadear comandos
# migrate roda primeiro — garante schema e tabelas atualizados
CMD ["sh", "-c", "python manage.py migrate && gunicorn Back_End.wsgi:application\n--bind 0.0.0.0:8000"]
```

7.2 Dockerfile do frontend — multi-stage build

O multi-stage build usa 2 stages: o primeiro instala o Node e compila o Vue, o segundo usa apenas o Nginx para servir os arquivos compilados. A image final nao contem Node — muito menor:

```
# front-end/Dockerfile

# STAGE 1: compilar o Vue com Node
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json .          # copia primeiro para aproveitar cache
RUN npm install
COPY . .
RUN npm run build              # gera a pasta dist/ com HTML/CSS/JS otimizados

# STAGE 2: servir com Nginx (sem Node na image final)
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html # copia apenas o dist
COPY nginx.conf /etc/nginx/conf.d/default.conf     # configuracao do proxy
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

7.3 nginx.conf — proxy reverso

O Nginx serve os arquivos estaticos do Vue e redireciona qualquer requisicao para /api/ para o container do Django. O try_files garante que o Vue Router funciona com history mode:

```
server {
    listen 80;

    # Serve os arquivos estaticos compilados do Vue
    location / {
        root /usr/share/nginx/html;
        index index.html;
        # Necessario para o Vue Router (history mode):
        # qualquer URL que nao exista como arquivo e redirecionada
        # para o index.html e o Vue trata o roteamento
        try_files $uri $uri/ /index.html;
    }

    # Proxy reverso: /api/ vai para o container do Django
    location /api/ {
        proxy_pass http://backend:8000;
        proxy_set_header Host          $host;
        proxy_set_header X-Real-IP      $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

7.4 docker-compose.yml — orquestracao completa

```
version: '3.9'
services:

    db:
        image: postgres:15-alpine
        container_name: revisoes_db
        restart: always
        env_file: .env
        environment:
            POSTGRES_DB:      ${POSTGRES_DB}
            POSTGRES_USER:    ${POSTGRES_USER}
            POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
        volumes:
            - postgres_data:/var/lib/postgresql/data # persiste entre restarts
        networks:
            - rede_app
        # sem 'ports' - banco nao fica exposto para o host

    backend:
```

```

build:
  context: ./Back_End
container_name: revisoes_backend
restart: always
env_file: .env
environment:
  DB_NAME:      ${POSTGRES_DB}
  DB_USER:      ${POSTGRES_USER}
  DB_PASSWORD:  ${POSTGRES_PASSWORD}
  DB_HOST:      db          # nome do container do banco
  DB_PORT:      5432
  SECRET_KEY:   ${DJANGO_SECRET_KEY}
  DEBUG:        ${DJANGO_DEBUG}
depends_on:
  - db          # aguarda o banco iniciar
ports:
  - '8000:8000'
networks:
  - rede_app

frontend:
  build:
    context: ./front-end
  container_name: revisoes_frontend
  restart: always
  ports:
    - '80:80'          # unica porta exposta ao usuario
  depends_on:
    - backend
  networks:
    - rede_app

volumes:
  postgres_data:          # volume nomeado - dados persistem

networks:
  rede_app:
    driver: bridge        # rede interna entre containers

```

7.5 Variaveis de ambiente

O projeto usa dois arquivos .env com propositos distintos:

Arquivo	Usado por	DB_HOST correto
Site_Carros/.env	Docker Compose	db (nome do container na rede)
Back_End/.env	Django sem Docker	localhost

```
# Site_Carros/.env (Docker Compose)
```

```
POSTGRES_DB=revisoes_db
POSTGRES_USER=jose_gabriel
POSTGRES_PASSWORD=sua_senha_aqui
DJANGO_SECRET_KEY=chave-longa-e-aleatoria
DJANGO_DEBUG=True
DJANGO_ALLOWED_HOSTS=localhost,127.0.0.1

# Back_End/.env (Django local sem Docker)
SECRET_KEY=chave-longa-e-aleatoria
DEBUG=True
DJANGO_ALLOWED_HOSTS=localhost,127.0.0.1
DB_NAME=revisoes_db
DB_USER=jose_gabriel
DB_PASSWORD=sua_senha_aqui
DB_HOST=localhost          # diferenca principal
DB_PORT=5432
```


8. Como Rodar o Projeto

8.1 Com Docker (recomendado)

Requisitos: Docker Engine e Docker Compose instalados.

12. Criar o arquivo Site_Carros/.env com as variaveis listadas na secao 7.5
13. Entrar na pasta raiz: `cd ~/Site_Carros`
14. Construir e subir os containers: `docker compose up --build -d`
15. Aguardar os logs confirmarem que o banco e o Django subiram: `docker compose logs backend --tail=20`
16. Opcional — popular com dados de teste: `docker compose exec backend python populate.py`
17. Acessar no navegador: `http://localhost`

8.2 Sem Docker — desenvolvimento local

Requisitos: Python 3.12, Node 20, PostgreSQL 15 instalados localmente.

Backend

18. Criar ambiente virtual: `python3 -m venv venv && source venv/bin/activate`
19. Instalar dependencias: `pip install -r Back_End/requirements.txt`
20. Criar banco, usuario e schema no PostgreSQL local
21. Criar Back_End/.env com `DB_HOST=localhost`
22. Rodar migrations: `cd Back_End && python3 manage.py migrate`
23. Iniciar servidor: `python3 manage.py runserver`

Frontend

24. `cd front-end`
25. Instalar dependencias: `npm install`
26. Criar .env.development com `VITE_API_URL=http://127.0.0.1:8000/api/`
27. Iniciar servidor de desenvolvimento: `npm run dev`
28. Acessar: `http://localhost:5173`

8.3 Comandos uteis do dia a dia

Comando	O que faz
<code>docker compose up -d</code>	Sobe todos os containers em background
<code>docker compose down</code>	Para e remove todos os containers
<code>docker compose down -v</code>	Para containers E apaga os volumes (reseta o banco)
<code>docker compose build --no-cache</code>	Reconstroi as images ignorando o cache
<code>docker compose ps</code>	Lista status de todos os containers
<code>docker compose logs backend --tail=30</code>	Ver os ultimos 30 logs do Django
<code>docker compose logs -f backend</code>	Acompanhar logs em tempo real
<code>docker compose restart backend</code>	Reinicia apenas o container do Django
<code>docker compose exec backend bash</code>	Abre terminal dentro do container Django
<code>docker compose exec backend python manage.py migrate</code>	Roda migrations manualmente
<code>docker compose exec backend python populate.py</code>	Popula o banco com dados de teste

9. Padroes e Decisoos Tecnicas

9.1 Backend

- `get_object_or_404` em todos os endpoints de detalhe — retorna 404 automaticamente em vez de 500
- `partial=True` em todos os PUTs — o frontend pode enviar apenas os campos alterados
- `select_related` em todas as listagens — elimina o problema N+1
- `prefetch_related` para relacoes aninhadas — usuarios com veiculos e revisoes em 3 queries
- Raw SQL apenas para window functions e CTEs — ORM para todo o resto
- `cursor.description` verificado antes de iterar — protege contra queries sem retorno de colunas
- Comentarios explicando o SQL equivalente em cada relatorio ORM
- Nomes de variaveis e funcoes em `snake_case` seguindo PEP 8

9.2 Frontend

- Composition API com script setup — padrao moderno e recomendado do Vue 3
- CSS modular em 6 arquivos — sem style scoped, sem CSS inline
- Variaveis CSS no `:root` — mudanca de tema em um unico lugar
- `computed()` para filtros locais — sem requisicoes extras a API
- `Promise.all` para carregamento inicial — todas as requisicoes em paralelo
- Atualizacao local da lista (`push/filter/splice`) — sem rebuscar toda a API
- `unshift()` para novos itens — aparece primeiro na lista
- Optional chaining (`?.`) nos modais — evita erro se o objeto for null
- `formatarData`, `formatarMoeda` e `formatarKm` centralizados — sem duplicacao
- `exibirMensagem` com `setTimeout` — feedback some automaticamente em 4 segundos

9.3 Docker e infraestrutura

- Multi-stage build no frontend — image final tem apenas Nginx, sem Node
- CMD com migrate antes do gunicorn — banco sempre atualizado ao subir
- Volume nomeado `postgres_data` — dados do banco persistem entre restarts
- Banco sem porta exposta no host — acessivel apenas pelos containers
- `env_file` no `docker-compose.yml` — credenciais fora do codigo versionado
- `.env` no `.gitignore` — credenciais nunca vao para o repositório
- `restart: always` — containers sobem automaticamente se o servidor reiniciar
- `depends_on: db` — Django aguarda o PostgreSQL antes de iniciar

Dados do ambiente de teste (populate.py)

6 proprietarios: 3 masculinos, 3 femininos, idades entre 28 e 47 anos

10 veiculos: 6 carros, 2 motos, 1 triciclo, 1 caminhao — marcas variadas

28 revisoes distribuidas entre 2023 e 2024 com custos e responsaveis reais

Intervalo medio entre revisoes varia de 45 a 120 dias por proprietario